# Non-Functional Requirements (NFR) Mapping

*Project: AI-Powered Expense Analyzer & Budget Assistant | Prepared by: Priyam Rai*

| NFR ID | NFR Name | Requirement Specification | Related Component(s) | Design / Implementation Approach | Verification Method |
|---|---|---|---|---|---|
| **NFR-01** | Performance / Response Time | Dashboard load < 2s; expense log ops < 500ms. | API Gateway, Expense Controller, `expenses` indexes | Apply DB query indexing on `user_id`, `category_id`, and `expense_date`. Utilize DB connection pooling. | Load testing using Artillery to check API endpoints latency under 100 concurrent requests. |
| **NFR-02** | Security | Secure all APIs via tokens; store credentials securely. | Authentication Services, JWT Router, `users` table | Hash passwords using bcrypt/Argon2. Use JWT token auth. Restrict admin roles via role ENUM ('user', 'admin'). | OWASP ZAP security scan; integration checks asserting HTTP 401 Unauthorized for missing tokens. |
| **NFR-03** | Availability | Target 99.9% monthly uptime (excluding maintenance). | API Server Deployment, MySQL Database (RDS) | Deploy nodes in multi-Availability Zones behind a Load Balancer with auto-scaling and auto-recovery policies. | Synthetic monitoring using Pingdom to scan status endpoint health every 60 seconds. |
| **NFR-04** | Prediction Accuracy | ML spend forecast model with MAPE < 15% on validation. | Spending Analysis & Prediction Module | Train forecasting models on expense history. Retrain monthly. Implement moving average fallback when history is sparse. | Automation scripts comparing predictions with holdout sets, scoring MAPE & RMSE. |
| **NFR-05** | Data Integrity | No currency rounding errors; no logical duplicate budgets. | Database Tier (MySQL), Budget/Expense Managers | Use exact `DECIMAL(10,2)` for money. Enforce unique constraint `uq_budget_user_month` and FK constraints with `ON DELETE RESTRICT`. | Integration unit tests forcing invalid duplicate budgets or negative expense amounts, verifying DB exceptions. |
| **NFR-06** | Scalability | Support up to 10,000 active concurrent users logging data. | API Gateway, App Nodes, MySQL Database Cluster | Stateless API server containers to allow horizontal scale-out. DB read-replicas and Redis caching for categories. | Stress/scaling tests verifying no resource bottlenecks at 10,000 virtual users concurrently active. |
| **NFR-07** | Maintainability | Develop modular codebase; unit test coverage >= 80%. | App Controllers/Services, CI/CD Pipeline | Follow SOLID design patterns. Strict separation of routing, business logic, and databases. Enforce standard code styles. | Integrate automated coverage reporting tools (e.g. Jest, pytest-cov, jacoco) into the CI/CD build scripts. |
| **NFR-08** | Reliability | Wrap db writes in transactions; gracefully handle notification dropouts. | DB Transaction Manager, Alert Notifier Module | Enforce SQL database ACID transactions for multi-table writes. Implement exponential back-off retries for alert triggers. | Inject network failure mock tests mid-transaction and confirm database state rolls back cleanly. |